

Tuyển tập Bài tập: Hàng đợi (Queue), Deque & Monotonic Queue

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Bảng Tổng hợp Bài tập Tuyển chọn

STT	Bài toán	Nền tảng	Độ khó	Kỹ thuật trọng tâm
1	Implement Queue using Stacks	LeetCode 232	★ Cơ bản	Mô phỏng FIFO bằng 2 Stacks $O(1)$ amortized
2	Implement Stack using Queues	LeetCode 225	★ Cơ bản	Mô phỏng LIFO bằng Queue xoay vòng
3	Counting Rooms	CSES 1192	★ Cơ bản	BFS loang đếm thành phần liên thông trên lưới 2D
4	Labyrinth	CSES 1193	★ Cơ bản	BFS tìm đường đi ngắn nhất & truy vết đường đi
5	Number of Recent Calls	LeetCode 933	★ Cơ bản	Queue lọc sự kiện hết hạn trong khoảng 3000ms
6	Design Circular Queue	LeetCode 622	★ Cơ bản	Cài đặt Hàng đợi vòng trên Mảng tĩnh (Circular Buffer)
7	Sliding Window Maximum	LeetCode 239	★★ Trung cấp	Monotonic Deque tìm Max cửa sổ K trong $O(N)$
8	Monsters	CSES 1194	★★ Trung cấp	Multi-source BFS (Loang đa nguồn đồng thời)
9	Labyrinth (0-1 BFS)	Codeforces 1063B	★★ Trung cấp	Thuật toán 0-1 BFS với <code>std::deque</code> tối ưu số bước rẽ
10	Chamber of Secrets	Codeforces 173B	★★ Trung cấp	Đổi hướng tia laser bằng đồ thị hai phía & 0-1 BFS

STT	Bài toán	Nền tảng	Độ khó	Kỹ thuật trọng tâm
11	Message Route	CSES 1667	★ ★ Trung cấp	BFS đường đi ngắn nhất không trọng số trên đồ thị vô hướng
12	Jump Game VI	LeetCode 1696	★ ★ Trung cấp	Tối ưu Quy hoạch động bằng Monotonic Deque $O(N)$
13	Constrained Subsequence Sum	LeetCode 1425	★ ★ ★ Nâng cao	DP dãy con có khoảng cách $\leq K$ + Monotonic Deque
14	Shortest Subarray with Sum at Least K	LeetCode 862	★ ★ ★ Nâng cao	Prefix Sums + Monotonic Deque tối ưu hai đầu $O(N)$
15	Array Stabilization by Deque	Codeforces 1579E2	★ ★ ★ Nâng cao	Tham lam Deque + Đếm nghịch thế bằng PBDS / BIT
16	KDIFF - Chênh lệch Cửa sổ	VNOI KDIFF	★ ★ ★ Nâng cao	Hai con trỏ + Hai Monotonic Deque (Max & Min)



HƯỚNG DẪN GIẢI CHI TIẾT & GỢI Ý CHIẾN THUẬT

1. Nhóm Bài tập Cơ bản (6 bài)

Bài 1: [LeetCode 232 - Implement Queue using Stacks](#)

- **Đề bài tóm tắt:** Cài đặt hàng đợi FIFO chỉ sử dụng hai ngăn xếp (Stack) chuẩn với độ phức tạp thời gian khấu hao $O(1)$ cho mọi thao tác.
- **Phân tích & Thuật toán:**
- Dùng `stack_in` nhận dữ liệu `push` và `stack_out` phục vụ `pop` và `peek`.
- Khi cần `pop` hoặc `peek`: nếu `stack_out` đang rỗng, trút toàn bộ phần tử từ `stack_in` sang `stack_out`. Thứ tự các phần tử được đảo ngược hai lần, khôi phục lại đúng nguyên lý FIFO.
- Mỗi phần tử được `push` và `pop` qua mỗi stack đúng 1 lần $\implies$ Khấu hao $O(1)$.

Bài 2: [LeetCode 225 - Implement Stack using Queues](#)

- **Đề bài tóm tắt:** Cài đặt ngăn xếp LIFO chỉ bằng cấu trúc dữ liệu Hàng đợi (Queue).
- **Phân tích & Thuật toán:**
- Khi `push(x)` vào queue: ghi nhận kích thước hiện tại $sz = q.size()$, đẩy x vào cuối queue, sau đó lặp sz lần lấy phần tử ở đầu đưa ra sau đuôi: `q.push(q.front()); q.pop();`.
- Lúc này, phần tử vừa thêm luôn nằm ở ngay đầu queue $\rightarrow$ thao tác `pop()` và `top()` đạt $O(1)$.

Bài 3: [CSES 1192 - Counting Rooms](#)

- **Đề bài tóm tắt:** Cho bản đồ mê cung kích thước $N \times M$ gồm các ô trống `.` và tường `#`. Đếm số lượng phòng độc lập (các vùng ô trống liên thông 4 hướng).
 - **Phân tích & Thuật toán:**
 - Duyệt qua từng ô (i, j) trên bản đồ: nếu ô là `.` và chưa được thăm, tăng biến đếm số phòng và kích hoạt BFS/DFS loang ra toàn bộ các ô kề của phòng đó.
 - Dùng một mảng đánh dấu `visited[N][M]` hoặc đổi trực tiếp ký tự `.` thành `#` để tránh duyệt lại.
 - Độ phức tạp: $O(N \times M)$ thời gian và không gian.
-

Bài 4: [CSES 1193 - Labyrinth](#)

- **Đề bài tóm tắt:** Tìm đường đi ngắn nhất từ ô bắt đầu `A` đến ô đích `B` trong mê cung $N \times M$. In ra độ dài đường đi và chuỗi ký tự các bước di chuyển (`U`, `D`, `L`, `R`).
 - **Phân tích & Thuật toán:**
 - BFS đảm bảo tìm được đường đi có số bước ít nhất trên đồ thị không trọng số.
 - Dùng mảng `parent[i][j]` lưu hướng di chuyển từ ô trước đó sang ô (i, j) (`U`, `D`, `L`, `R`).
 - Khi BFS chạm tới đích `B`, dừng thuật toán và đi ngược theo mảng `parent` từ `B` về `A`, sau đó đảo ngược chuỗi để có thứ tự đúng.
-

Bài 5: [LeetCode 933 - Number of Recent Calls](#)

- **Đề bài tóm tắt:** Thiết kế bộ đếm nhận các yêu cầu tại thời điểm t . Trả về số lượng yêu cầu xảy ra trong khoảng $[t - 3000, t]$.
 - **Phân tích & Thuật toán:**
 - Mỗi khi nhận được `ping(t)`, đẩy t vào cuối `queue<int> q`.
 - Trong khi `q.front() < t - 3000`, loại bỏ phần tử đầu hàng đợi `q.pop()`.
 - Kết quả chính là `q.size()`. Thời gian khấu hao $O(1)$ mỗi lệnh gọi.
-

Bài 6: [LeetCode 622 - Design Circular Queue](#)

- **Đề bài tóm tắt:** Thiết kế hàng đợi vòng tròn (Circular Queue) có kích thước cố định K trên mảng, hỗ trợ `enqueue`, `dequeue`, `Front`, `Rear`, `isEmpty`, `isFull` trong $O(1)$ không cấp phát động.
 - **Phân tích & Thuật toán:**
 - Dùng mảng `data[K]`, biến `head = 0`, `tail = 0`, và `count = 0`.
 - Khi thêm: `data[tail] = val; tail = (tail + 1) % K; ++count;`
 - Khi xóa: `head = (head + 1) % K; --count;`
-

2. Nhóm Bài tập Trung cấp (6 bài)

Bài 7: [LeetCode 239 - Sliding Window Maximum](#)

- **Đề bài tóm tắt:** Cho mảng N phần tử và kích thước cửa sổ K . Tìm giá trị lớn nhất trong mọi cửa sổ trượt kích thước K ($N \leq 10^5$).
 - **Phân tích & Thuật toán:**
 - Dùng **Monotonic Deque** lưu trữ chỉ số các phần tử sao cho giá trị $A[\text{index}]$ giảm dần nghiêm ngặt.
 - Với mỗi i :
 1. Xóa phần tử hết hạn: `if (!dq.empty() && dq.front() <= i - K) dq.pop_front();`
 2. Duy trì đơn điệu giảm: `while (!dq.empty() && A[dq.back()] <= A[i]) dq.pop_back();`
 3. Đẩy i vào cuối: `dq.push_back(i);`
 4. Ghi nhận max tại `A[dq.front()]` khi $i \geq K - 1$.
 - Độ phức tạp: $O(N)$ thời gian, $O(K)$ bộ nhớ phụ trợ.
-

Bài 8: [CSES 1194 - Monsters](#)

- **Đề bài tóm tắt:** Bạn ở ô A , trong mê cung có nhiều quái vật M . Mỗi bước đi, bạn và tất cả quái vật có thể di chuyển 1 ô theo 4 hướng. Tìm đường thoát ra biên mê cung sao cho không bao giờ bị quái vật bắt kịp.
 - **Phân tích & Thuật toán:**
 - **Kỹ thuật Multi-source BFS:**
 - Bước 1: Đẩy tất cả các ô xuất phát của quái vật M vào một queue BFS chung để tính mảng `monster_time[i][j]` (thời điểm sớm nhất quái vật đến được từng ô).
 - Bước 2: Chạy BFS từ vị trí của bạn A , chỉ đi vào ô (u, v) tại thời điểm t nếu $t < \text{monster_time}[u][v]$.
 - Nếu chạm vào bất kỳ ô biên nào của mê cung, bạn thoát thành công!
 - Độ phức tạp: $O(N \times M)$.
-

Bài 9: [Codeforces 1063B - Labyrinth \(0-1 BFS\)](#)

- **Đề bài tóm tắt:** Từ ô xuất phát (r, c) , bạn có thể di chuyển lên/xuống không giới hạn, nhưng chỉ được di chuyển sang trái tối đa X lần và sang phải tối đa Y lần. Đếm số ô có thể đi tới.
 - **Phân tích & Thuật toán:**
 - Trọng số khi đi lên/xuống là 0 bước rẽ trái/phải.
 - Trọng số khi đi sang trái là 1 bước rẽ trái.
 - Trọng số khi đi sang phải là 1 bước rẽ phải.
 - Sử dụng **0-1 BFS** với `std::deque`:
 - Đi lên/xuống: đẩy vào đầu Deque (`push_front`).
 - Đi sang trái/phải: đẩy vào cuối Deque (`push_back`).
 - Độ phức tạp: $O(N \times M)$ tối ưu tuyệt đối, không cần dùng Dijkstra.
-

Bài 10: [Codeforces 173B - Chamber of Secrets](#)

- **Đề bài tóm tắt:** Một chùm laser đi từ hàng 1 sang hàng N . Trong mê cung có các gương phản xạ `#` tại giao điểm hàng và cột. Tìm số gương tối thiểu cần dùng để bẻ hướng tia laser tới đích.
- **Phân tích & Thuật toán:**
- Mô hình hóa đồ thị: Mỗi hàng và mỗi cột là một đỉnh (gồm N đỉnh hàng và M đỉnh cột).
- Một gương `#` tại ô (i, j) tạo ra một cạnh vô hướng giữa đỉnh hàng i và đỉnh cột j với trọng số 1.
- Tìm đường đi ngắn nhất từ hàng N tới hàng 1 bằng 0-1 BFS hoặc BFS thường trên đồ thị có $N + M$ đỉnh và $2K$ cạnh.

Bài 11: [CSES 1667 - Message Route](#)

- **Đề bài tóm tắt:** Cho mạng máy tính gồm N máy và M dây nối hai chiều. Tìm đường truyền tin ngắn nhất từ máy 1 tới máy N và in ra dãy các máy trên đường đi đó.
- **Phân tích & Thuật toán:**
- Thuật toán BFS tiêu chuẩn: duy trì mảng `dist[]` và mảng `parent[]` để truy vết.
- Nếu `dist[N] == INF`, in `IMPOSSIBLE`. Ngược lại, truy vết từ N về 1 bằng vòng lặp qua `parent`.

Bài 12: [LeetCode 1696 - Jump Game VI](#)

- **Đề bài tóm tắt:** Bắt đầu từ chỉ số 0, tại mỗi bước được nhảy về phía trước từ 1 đến K bước. Điểm số nhận được là tổng giá trị các ô đặt chân tới. Tìm điểm số lớn nhất để tới được chỉ số $N - 1$.
- **Phân tích & Thuật toán:**
- Công thức DP:

$$DP[i] = A[i] + \max_{i-K \leq j < i} DP[j]$$

- Duy trì $\max DP[j]$ trong cửa sổ trượt kích thước K bằng **Monotonic Deque** giảm dần.
- Giảm độ phức tạp từ $O(N \cdot K)$ xuống $O(N)$ tuyến tính.

3. Nhóm Bài tập Nâng cao (4 bài)

Bài 13: [LeetCode 1425 - Constrained Subsequence Sum](#)

- **Đề bài tóm tắt:** Tìm tổng lớn nhất của dãy con khác rỗng sao cho với mọi cặp phần tử liên tiếp được chọn $A[i], A[j]$ ($i < j$), ta có $j - i \leq K$.
- **Phân tích & Thuật toán:**
- Gọi $DP[i]$ là tổng lớn nhất của dãy con kết thúc tại chỉ số i :

$$DP[i] = A[i] + \max(0LL, \max_{i-K \leq j < i} DP[j])$$

- Dùng Monotonic Deque duy trì giá trị lớn nhất của $DP[j]$ trong phạm vi $j \in [i - K, i - 1]$.
- Độ phức tạp: $O(N)$ thời gian, $O(K)$ bộ nhớ phụ trợ.

Bài 14: [LeetCode 862 - Shortest Subarray with Sum at Least K](#)

- **Đề bài tóm tắt:** Cho mảng N phần tử (có thể âm). Tìm độ dài mảng con liên tiếp ngắn nhất có tổng $\geq K$. Nếu không có, trả về -1 .
- **Phân tích & Thuật toán:**
- Tính mảng tiền tố $P[i] = \sum_{j=0}^{i-1} A[j]$ với $P[0] = 0$. Điều kiện tổng mảng con $[j, i - 1] \geq K$ tương đương:

$$P[i] - P[j] \geq K \iff P[j] \leq P[i] - K \quad (j < i)$$

- Duy trì một Deque lưu các chỉ số j sao cho $P[j]$ tăng dần nghiêm ngặt:
 1. Khi xét đến i : kiểm tra từ đầu deque `while (!dq.empty() && P[i] - P[dq.front()] >= K)`, cập nhật `min_len = min(min_len, i - dq.front())` và `dq.pop_front()` (vì j này đã tìm được độ dài kết thúc tại i , các chỉ số sau i nếu ghép với j sẽ chỉ tạo ra độ dài lớn hơn).
 2. Trước khi thêm i , loại bỏ các j ở cuối deque có $P[j] \geq P[i]$ (vì i vừa có giá trị P nhỏ hơn, vừa có chỉ số lớn hơn $\rightarrow$ ưu thế hơn j).
- Độ phức tạp: $O(N)$ thời gian.

Bài 15: [Codeforces 1579E2 - Array Stabilization by Deque](#)

- **Đề bài tóm tắt:** Cho mảng A . Với mỗi phần tử, bạn được chọn chèn nó vào đầu hoặc cuối của một Deque. Tìm số lượng nghịch thế nhỏ nhất có thể tạo ra trong Deque cuối cùng.
- **Phân tích & Thuật toán:**
- Khi chèn phần tử x :
 - Nếu chèn vào đầu: tạo thêm số nghịch thế bằng số lượng phần tử hiện có trong Deque nhỏ hơn x (`count(< x)`).
 - Nếu chèn vào cuối: tạo thêm số nghịch thế bằng số lượng phần tử hiện có trong Deque lớn hơn x (`count(> x)`).
- Chiến lược tham lam: So sánh `count(< x)` và `count(> x)`, chọn bên tạo ít nghịch thế hơn.
- Sử dụng GNU PBDS `ordered_set` hoặc Fenwick Tree để đếm số lượng phần tử trong $O(\log N)$.
- Tổng độ phức tạp: $O(N \log N)$.

Bài 16: [VNOI - KDIFF \(Chênh lệch Cửa sổ\)](#)

- **Đề bài tóm tắt:** Cho mảng N phần tử và số K . Tìm độ dài lớn nhất của một đoạn con sao cho chênh lệch giữa phần tử lớn nhất và nhỏ nhất trong đoạn không vượt quá K : $\max(A[L..R]) - \min(A[L..R]) \leq K$.
- **Phân tích & Thuật toán:**
- Kỹ thuật Hai con trỏ (Two Pointers) kết hợp **Hai Monotonic Deques**:
 - `dq_max`: Duy trì phần tử lớn nhất trong cửa sổ hiện tại tại $[L, R]$.
 - `dq_min`: Duy trì phần tử nhỏ nhất trong cửa sổ hiện tại tại $[L, R]$.
- Khi mở rộng con trỏ phải R : cập nhật cả hai deque.

- Nếu `A[dq_max.front()] - A[dq_min.front()] > K` : tăng con trỏ trái L , loại bỏ các phần tử hết hạn khỏi cả hai deque cho tới khi điều kiện được thỏa mãn.
 - Độ phức tạp: $O(N)$ tuyến tính tuyệt đối.
-



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.